

Circles Renderer - Parallel Rendering with OpenMP

Matteo Lotti
7154910

matteo.lotti3@edu.unifi.it

Abstract

This project focuses on the parallel implementation of a simple rendering algorithm using OpenMP in C++. Specifically, the renderer has to draw semi transparent circles, each defined by its center position, radius, depth (z-value), color, and transparency (alpha). The aim is to implement a parallel version and test its potential benefits in terms of execution times with respect to the sequential version.

1. Introduction

Speaking about image rendering, even when dealing with simplified models, such as two-dimensional circles, the computational effort can be significant due to the large number of pixels and objects involved. That is the reason why this case represent a good scenario to be tested with a parallel implementation.

The goal of this project is to implement a simple rendering system in C++ that generates an image composed of circles, each defined by the position of its center in space (x , y), a radius, a color (RGB), an order along Z axis and a transparency level (alpha). The algorithm determines, for each pixel, whether it belongs to one or more circles and updates its color using alpha blending. The depth coordinate (z) is used to establish the correct ordering of circles, ensuring that objects with higher z -value are rendered in front of those with lower z -value. Figure 1 shows an example of circles rendering, in which you can see how transparency and alpha blending works. To improve performance, the application exploits parallelism with OpenMP, distributing the workload across multiple threads; then a comparison with the sequential version is set up. In the following sections, the structure of the application will be described in detail, including the data representation of circles and the image buffers, the rendering algorithm, and the use of OpenMP directives for parallel execution.



Figure 1. Rendering example

1.1. The classes

In order to represent the circles, two approaches were adopted to define the structure used:

- Array of Structure (AoS)
- Structure of Array (SoA)

The goal is to compare these approaches to find the best one for this kind of experiment in terms of execution times and speedup. As we mentioned before, each circle is represented by the 2D position of its center, the radius, a Z -value and a transparency level. AoS version uses a *Circle* class (listing 1), while SoA version is build on a *CircleSoA* class (listing 2). Those two kind of structures are exploited by classes that implements the various rendering pipelines; thos classes extends the *Utils* class. Generally speaking, all the *Utils* classes present the same features:

- They have three 2D vectors representing the RGB values for each pixel.

- They randomly generate circles (and all their features) that will be rendered.
- They implement the rendering method, either a sequential or parallel version.
- They save the image in PPM format.

```

1 class Circle {
2 private:
3     double x, y, z, r; //center, z-value
4         and radius
5     double red, green, blue; //color
6 public:
7     Circle(double x, double y, double z,
8             double r, double red, double green,
9             double blue):
10        x(x), y(y), z(z), r(r), red(red),
11        green(green), blue(blue) {}
12    ~Circle() = default;
13    //...
14    // getters and setters
15    //...
16 }

```

Listing 1. AoS version of circles

```

1 class CircleSoA {
2 private:
3     double* x;
4     double* y;
5     double* z;
6     double* r;
7     double* red;
8     double* green;
9     double* blue;
10    int size;
11 public:
12    CircleSoA(int size) : size(size) {
13        //...initializations...
14    }
15    //...
16    // getters and setters
17    //...
18 }

```

Listing 2. SoA version of circles

In the next section all the implementations will be discussed, as well as the choices made for parallelization.

2. Implementation of the rendering pipeline

The rendering pipeline is therefore built by:

- A *generateCircles()* method, responsible for the random generation of features.
- A *renderCircles()* method that represents the core of the project, which computes the blended color of each pixel, taking into account the Z-value of each circle.
- A *savePPM()* method to convert RGB vectors to PPM format images

In the subsection will be shown the details of implementations for the sequential version and the choices taken for the OpenMP parallel version.

2.1. Sequential version

2.1.1 Circles Generation

As we can see from listing 3 and 4, generation is made by sampling from a uniform distribution each feature of the circle. Specifically, circle's centre could be everywhere in the image, the Z-value is a double between 0 and 1, as well as RGB values, while the radius is a double between 10 and 100.

```

1 void CircleUtils::generateCircles() {
2
3     std::vector<Circle*> circles;
4
5     std::default_random_engine generator;
6     std::uniform_real_distribution<double>
7         distributionX(0.0, width);
8     std::uniform_real_distribution<double>
9         distributionY(0.0, height);
10    std::uniform_real_distribution<double>
11        distributionZ(0.0, 1.0);
12    std::uniform_real_distribution<double>
13        distributionR(10.0, 100.0);
14    std::uniform_real_distribution<double>
15        distributionRGB(0.0, 1.0);
16
17    for (int i = 0; i < this->n; i++) {
18        double x = distributionX(generator);
19        ;
20        double y = distributionY(generator);
21        ;
22        double z = distributionZ(generator);
23        ;
24        double r = distributionR(generator);
25        ;
26        double red = distributionRGB(
27            generator);
28        double green = distributionRGB(
29            generator);
30        double blue = distributionRGB(
31            generator);
32    }
33 }

```

```

20         circles.push_back(new Circle(x, y,
21             z, r, red, green, blue));
22     }
23     this->circles = circles;
24 }

```

Listing 3. AoS Sequential Cirlces generation

```

1 void CircleSoAUtils::generateCircles() {
2     std::default_random_engine generator;
3     std::uniform_real_distribution<double>
4         distributionX(0.0, width);
5     std::uniform_real_distribution<double>
6         distributionY(0.0, height);
7     std::uniform_real_distribution<double>
8         distributionZ(0.0, 1.0);
9     std::uniform_real_distribution<double>
10        distributionR(10.0, 100.0);
11    std::uniform_real_distribution<double>
12        distributionRGB(0.0, 1.0);
13
14    for (int i = 0; i < this->n; i++) {
15        circles->setX(i, distributionX(
16            generator));
17        circles->setY(i, distributionY(
18            generator));
19        circles->setZ(i, distributionZ(
20            generator));
21        circles->setR(i, distributionR(
22            generator));
23        circles->setRed(i, distributionRGB(
24            generator));
25        circles->setGreen(i,
26            distributionRGB(generator));
27        circles->setBlue(i, distributionRGB(
28            generator));
29    }
30 }

```

Listing 4. SoA Sequential Cirlces generation

2.1.2 Rendering

This is the main part of the experiment, as in this method each pixel's color is computed. As you can see from listings 5 and 6, the initial step is sorting circles with respect to their Z-values, so as to render circles with higher Z on the foreground. Then, we have a double loop that let us iterate on each pixel of the image. For each pixel we compute its center; then we loop on the sorted circles vector to verify if the center of the pixel is inside the circle: if this is the case, we update the RGB values of the pixel eith the alpha blending technique.

```

1 void CircleUtils::renderCircles() {
2     std::vector<Circle*> sortedCircles =
3         this->circles;

```

```

3     std::sort(sortedCircles.begin(),
4         sortedCircles.end(), [](Circle* a,
5         Circle* b) {
6         return a->getZ() < b->getZ();
7     });
8     for(int y = 0; y < this->height; y++) {
9         for(int x = 0; x < this->width; x
10            ++){
11             //computing the pixel's center
12             double px = x + 0.5;
13             double py = y + 0.5;
14
15             double r, g, b = 1.0;
16             for(int i=0; i<sortedCircles.
17                 size(); i++) {
18                 Circle* circle =
19                     sortedCircles[i];
20                 if(isInsideCircleAoS(px, py
21                     , circle)) {
22                     r = alpha * circle->
23                         getRed() + (1 -
24                         alpha) * r;
25                     g = alpha * circle->
26                         getGreen() + (1 -
27                         alpha) * g;
28                     b = alpha * circle->
29                         getBlue() + (1 -
30                         alpha) * b;
31                 }
32             }
33             imgR[x][y] = r;
34             imgG[x][y] = g;
35             imgB[x][y] = b;
36         }
37     }

```

Listing 5. AoS Circles Rendering

```

1 \begin{lstlisting}[capiotn={AoS Circles
2     Rendering}, label={AoS_seq_rendering}]
3 void CircleUtils::renderCircles() {
4     std::vector<Circle*> sortedCircles =
5         this->circles;
6     std::sort(sortedCircles.begin(),
7         sortedCircles.end(), [](Circle* a,
8         Circle* b) {
9         return a->getZ() < b->getZ();
10    });
11    for(int y = 0; y < this->height; y++) {
12        for(int x = 0; x < this->width; x
13            ++){
14            double px = x + 0.5;
15            double py = y + 0.5;
16
17            double r, g, b = 1.0;
18            for(int i=0; i<sortedCircles.
19                size(); i++) {
20                Circle* circle =

```

```

15         sortedCircles[i];
16         if(isInsideCircleAoS(px, py
17             , circle)) {
18             r = alpha * circle->
19                 getRed() + (1 -
20                 alpha) * r;
21             g = alpha * circle->
22                 getGreen() + (1 -
23                 alpha) * g;
24             b = alpha * circle->
25                 getBlue() + (1 -
26                 alpha) * b;
27         }
28     }
29     imgR[x][y] = r;
30     imgG[x][y] = g;
31     imgB[x][y] = b;
32 }
33 }

```

Listing 6. SoA Circles Rendering

Alpha Blending Alpha Blending is a technique used in computer graphics to combine the color of a foreground object with the color of the background, based on the object's transparency level. The transparency is controlled by a parameter called alpha (α), which generally ranges between 0 and 1. When α approaches to 0, the object get transparent; on the other side, when α is near to 1, the object is more opaque. The blending is computed for each color channel (red, green, blue) using the formula:

$$C_{out} = \alpha \cdot C_{foreground} + (1 - \alpha) \cdot C_{background}$$

In the implementation, α is fixed and set to 0.5.

2.1.3 Image Saving in PPM format

This step only regards the output of the image file; PPM (Portable Pixmap) is a simple image file format that stores pixel data as plain text or binary, with each pixel defined by its red, green, and blue (RGB) values.

2.2. Parallel version

Since the parallel version is done by using OpenMP, the structure of the code does not change significantly with respect to the sequential one, as generally only the OMP directives are inserted. Some short listings will be shown and the choices made for the parallelized version will be described.

2.2.1 Circles generation

The method responsible for circles generation has also been parallelized with OpenMP directives. As we can see from

listing 7, valid for both AoS and SoA version as they are analogous, the *generator* is declared outside the parallel section and has been privatized, as it is accessed multiple times. Then, inside the parallel section the for loop is parallelized and scheduled as *static*, since the iterations should be similar, so the work load is divided equally between threads.

```

1 void CircleUtilsParallel::
2     generateCircles() {
3         std::default_random_engine generator;
4         #pragma omp parallel num_threads(
5             omp_get_num_threads()) private(
6             generator)
7         {
8             //...same a sequential...
9             #pragma omp for schedule(static)
10            for (int i = 0; i < this->n; i++) {
11                //...same a sequential...

```

Listing 7. AoS/SoA Parallel Circles generation

2.2.2 Rendering

The core rendering part is mainly built by for loops; we are going to analyze them in order to evaluate which had to be parallelized. Taking into account the nested loop that iterates on each pixel of the image, it has been parallelized with the directive *collapse(2)* in order to apply the OMP for not only to the outer loop but also to the inner one. Then the OMP schedule has been set to static, since the same operations have to be done for every pixel, so we can distribute the work load equally among threads. Then the *sortedCircles* vector has been privatized, since it has to be accessed frequently by each thread; specifically the *firstprivate* directive has been used so we have an initialized copy in each thread. The for loop iterating on the sorted circles has not been parallelized, since for each pixels we have to verify if it is inside each circle, evaluating them in the sorted order, so we can then compute the alpha blending with the correct order. What has to be pointed out is that the instructions that update the *imgR*, *imgG* and *imgB* vectors could be vulnerable to false sharing in the neighboring indexes between two threads' blocks; however, even if there is some false sharing, taking into account the sizes of the vectors, a small percentage of objects could suffer from this problem and seriously affect the execution time. In particular, since the image size is 800x800, each vector has 640000 elements, so when we use 20 threads at most 0.003% of the elements may be at risk, causing, with high probability, an insignificant waste of time.

```

1 void CircleUtilsParallel::renderCircles
2     () {

```

```

2      std::vector<Circle *> sortedCircles =
          this->circles;
3      std::sort(sortedCircles.begin(),
          sortedCircles.end(), [](Circle *a,
          Circle *b) {
4          return a->getZ() < b->getZ();
5      });
6
7      #pragma omp parallel for collapse(2)
          schedule(static) firstprivate(
          sortedCircles)
8      for (int y = 0; y < this->height; y++)
9      {
10         //...same a sequential...
11         imgR[x][y] = r;
12         imgG[x][y] = g;
13         imgB[x][y] = b;
14     }
15 }

```

Listing 8. AoS/SoA Parallel Circles rendering

3. Experiments and Results

Testbed features Experiments have been run on an AMD Ryzen 7 5700G CPU, which features 8 core and 16 threads and operates at 3.8 GHz by default, but can boost up to 4.6 GHz, depending on the workload.

3.1. Experiments setup

The renderer has been tested populating image with 800x800 resolution, evaluating the execution times for both sequential and parallel version and both AoS and SoA approach. Then, times have been calculating spawning 500, 1000 and 2000 circles, averaging the results on 10 runs. Regarding the parallel benchmark, it has been tested on an increasing number of threads, going from 2 to 20. As we mentioned that our testbed features 16 threads, we expect a performance drop at that point due to overhead caused by a higher number of threads.

3.2. Results

In tables 1, 2, 3 and 4 we can take a view at the sequential and parallel execution times. As also shown by figures 2, 3 and 4, AoS is faster in terms of execution times, but the quite significant difference in the sequential version is less strong when parallelizing with more threads. Then, if we take a look at the speedup results, in tables 5, 6 and 7, and in figures 5, 6 and 7, we can see that AoS version has generally a better speedup, but both versions have an increasing but sublinear trend, as the number of threads grows up; moreover, plots for both execution times and speedup show

that the optimal number of thread is 16, as we mentioned before in 3 speaking about the testbed features.

4. Conclusions

Ultimately, our comparative analysis of the Array of Structures (AoS) and Structure of Arrays (SoA) data layouts revealed that AoS outperforms SoA in both execution time and speedup. This superior performance can be attributed to the principle of spatial locality and the efficient use of the CPU’s cache hierarchy. In our specific application, when renderer involves the step of checking if a pixel falls within a circle, the computation for each circle requires simultaneous access to all of its attributes (for example, center coordinates and radius). With the AoS layout, all of these attributes for a single circle are stored contiguously in memory. When a thread accesses one attribute, the entire circle’s data is loaded into the fast CPU cache in a single operation. This ensures that subsequent accesses to the other attributes of the same circle are satisfied by the cache, leading to a high cache hit rate and significantly reducing costly memory fetches. In contrast, the SoA layout stores different attributes in separate, non-contiguous arrays. This forces the CPU to perform multiple memory accesses to fetch all the necessary data for a single circle, leading to more cache misses and slower execution.

Despite the superior performance of AoS, our parallel implementation of the algorithm exhibits a sublinear speedup. This means that the speedup does not scale linearly with the number of threads, ultimately plateauing and even decreasing at higher thread counts, when deploying more than 16 threads. This behavior could be caused by multiple factors, including the overhead of thread creation and synchronization, as well as possible false sharing issues, as briefly mentioned in 2.2.2. However, even with this limitation, our parallel implementation with an optimal number of threads provides a substantial performance gain. For instance, in the case of 2000 circles, the AoS parallel implementation successfully reduced the execution time from approximately 14 seconds down to 1.7 seconds, demonstrating the practical benefits of parallelization despite the sublinear scaling.

5. Appendix - Tables and Plots

Number of Circles	AoS Time (s)	SoA Time (s)
500	3.6438	4.5582
1000	7.3296	9.1344
2000	14.8122	18.7308

Table 1. Sequential Execution Times (s)

Number of threads	AoS time (s)	SoA time (s)
2	1.99882	2.74391
4	1.11725	1.55668
6	0.78733	1.12402
8	0.63252	0.89121
10	0.53075	0.76014
12	0.49769	0.68765
14	0.45293	0.63817
16	0.43634	0.63439
18	0.47725	0.68030
20	0.45670	0.65519

Table 2. Parallel Execution Times (s) - 500 Circles

Number of threads	AoS time (s)	SoA time (s)
2	3.90494	5.32043
4	2.27463	3.20602
6	1.60321	2.29256
8	1.27666	1.79069
10	1.09172	1.52759
12	0.97378	1.38245
14	0.88958	1.26550
16	0.83656	1.22224
18	0.94695	1.32627
20	0.92347	1.29474

Table 3. Parallel Execution Times (s) - 1000 Circles

Number of threads	AoS time (s)	SoA time (s)
2	7.93121	11.05520
4	4.45636	6.36125
6	3.19305	4.50253
8	2.52470	3.63357
10	2.17975	3.13521
12	1.95573	2.77415
14	1.76858	2.56710
16	1.72546	2.43934
18	1.84293	2.58998
20	1.81602	2.60671

Table 4. Parallel Execution Times (s) - 2000 Circles

Number of threads	AoS Speedup	SoA Speedup
2	1.82	1.66
4	3.26	2.93
6	4.63	4.06
8	5.76	5.11
10	6.87	6.00
12	7.32	6.63
14	8.05	7.14
16	8.35	7.19
18	7.63	6.70
20	7.98	6.96

Table 5. Parallel Speedup - 500 Circles

Number of threads	AoS Speedup	SoA Speedup
2	1.88	1.72
4	3.22	2.85
6	4.57	3.98
8	5.74	5.10
10	6.71	5.98
12	7.53	6.61
14	8.24	7.22
16	8.76	7.47
18	7.74	6.89
20	7.94	7.05

Table 6. Parallel Speedup - 1000 Circles

Number of threads	AoS Speedup	SoA Speedup
2	1.87	1.69
4	3.32	2.94
6	4.64	4.16
8	5.87	5.15
10	6.80	5.97
12	7.57	6.75
14	8.38	7.30
16	8.58	7.68
18	8.04	7.23
20	8.16	7.19

Table 7. Parallel Speedup - 2000 Circles

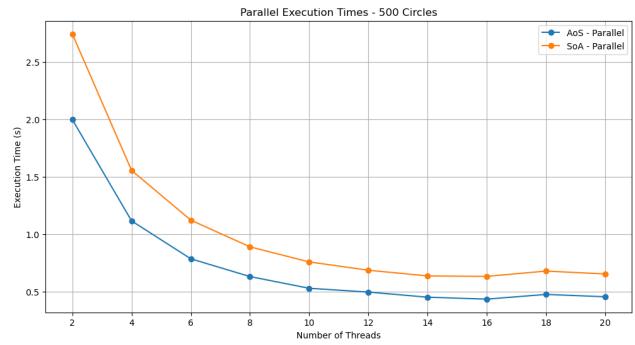


Figure 2. Parallel Execution Times - 500 Circles

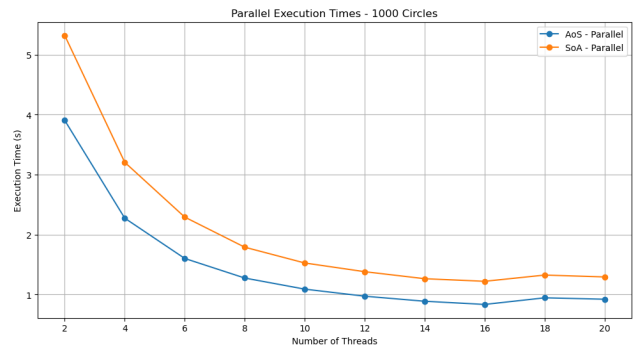


Figure 3. Parallel Execution Times - 1000 Circles

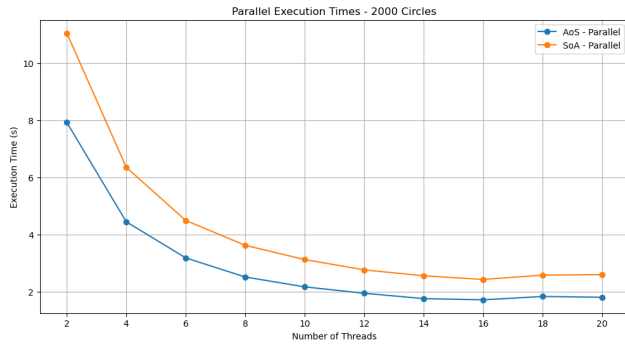


Figure 4. Parallel Execution Times - 2000 Circles

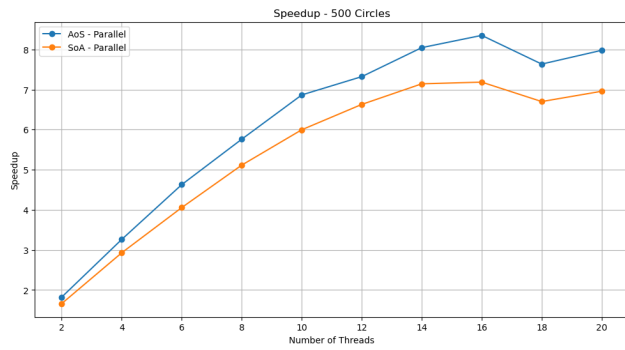


Figure 5. Parallel Speedup - 500 Circles

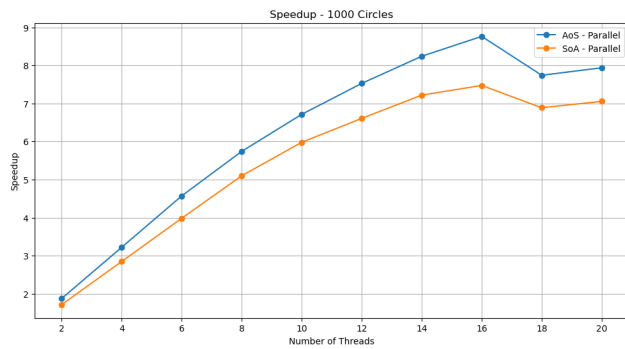


Figure 6. Parallel Speedup - 1000 Circles

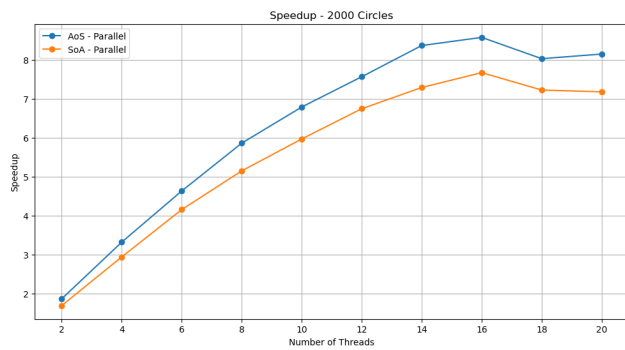


Figure 7. Parallel Speedup - 2000 Circles